Robotaxi 乘客反馈管理平台复盘文档

1. 需求拆解思路

我对这道题的理解，不是单纯做一个“反馈列表 + 看板 + AI总结”的展示型项目，而是要在有限时间内，搭建一个**能够体现业务理解、产品拆解能力和 AI 协作能力**的可用平台。题目要求的核心模块包括反馈列表与管理、数据统计仪表盘、AI 智能分析，同时强调要通过这个过程体现“用 AI 工具把模糊需求变成可用产品”的能力。

在正式设计前，我先补了一步业务理解。由于滴滴自动驾驶真实用户评价较少，我先去社交平台搜索了“萝卜快跑”的用户评价，观察真实用户会抱怨什么、表达方式是什么、哪些问题会被频繁提及。我的结论是：高频差评主要集中在接驾体验和路线规划两个方向。前者包括上下车点不方便、站点太少、预计到达时间不准、等待时间过长；后者包括为了遵守规则或绕行导致行驶路线过长、耗时明显高于预期。除此之外，我还注意到一个很关键的点：自动驾驶作为新技术，用户第一次使用时的理解成本很高，很多负面反馈并不是“车开得不好”，而是“我不知道怎么用”“为什么会停在这里”“车为什么自己取消了订单”。所以我把“新用户引导”也作为一个独立且重要的分析维度。

在此基础上，我没有把平台用户理解成泛用户，而是先明确它的第一目标用户是**运营团队**。围绕运营的工作目标，我把需求进一步拆成五类：第一，快速识别线上问题及其影响范围；第二，判断哪些问题值得优先投入资源处理；第三，监控是否存在异常情况或集中爆发的问题；第四，跟踪问题是否真正闭环；第五，为运营的日报、周报、月报提供总结材料。这一步对后续产品结构很关键，因为它决定了平台不只是“存反馈”，而是要同时服务于**处理、判断和汇报**三个场景。

基于这个理解，我把需求主干收敛到了三层。第一层是题目明确要求的基础能力，也就是反馈列表、详情、筛选、趋势看板和 AI 分类/摘要/建议。第二层是为了让平台更贴近运营实际工作而补充的能力，其中最重要的是工单体系，因为运营不能只“看到问题”，还需要建单、跟踪状态、看 SLA、知道谁在处理。第三层才是进阶能力，比如问题聚类、优先级辅助判断、轻量舆情输入等。

在功能取舍上，我做了几轮调整。最开始我确实想把很多“更高级”的能力一起做进去，比如情绪识别参与优先级判断、社媒自动抓取、A/B Test 支持、异常检测等。但在后续梳

理中，我逐步做了收敛。首先，我把优先级判断里基于情绪的排序弱化了，因为在自动驾驶场景里，真正应该优先处理的不是“说得最激烈”的用户，而是**安全相关、故障相关、可能升级成事故或严重投诉**的问题。其次，我原本设想把 A/B Test 做成平台能力，但后来发现这会把平台边界拉得过大，更合理的方式是把实验信息作为可筛选维度，而不是在反馈平台里做一个完整实验平台。第三，我一开始希望自动抓取小红书、微博、抖音等社媒反馈，但在实际验证中发现平台抓取限制非常强，短时间内做一个稳定可靠的爬虫既不现实，也会把项目重点从“反馈管理平台”转向“舆情采集系统”。所以最终我保留了社媒反馈这个来源，但把实现方式收敛为**轻量导入或接入已有信号源**，而不是自建抓取。关于这些功能边界、SLA、1:1 反馈-工单关系、轻量社媒方案、仪表盘和 AI 分析范围，聊天记录里也经历了多轮确认和修正。

最终，我把 V1 的重点定义为：在题目要求之上，增加一个轻量但完整的工单闭环，让平台从“能看”升级到“能处理”；同时用 AI 做分类、摘要和建议，让它真正体现 AI PM 的能力，而不是只做静态页面。这个取舍背后的逻辑是：48 小时内，最重要的不是功能堆砌，而是把主链路做对、边界讲清楚、重点做实。

---

2. 技术选型理由

一开始在技术方案上，我和 AI 讨论过偏工程化的一套方案，包括 React + TypeScript 前端，以及 Go 或 Java 后端，数据库使用 PostgreSQL，并在后续接入 Redis、Kafka、Elasticsearch、pgvector 等能力。这个方案从长期演进角度是合理的，也更符合一个正式内部系统的复杂度。聊天记录里最初的技术规划就包含了这些方向，例如用 Redis 做 SLA 扫描、用 Kafka 做 AI 流水线、用 pgvector 做相似反馈检索和聚类。

但在真正开始实现后，我做了一个很现实的调整：由于当前环境里没有 Go，也没有可直接使用的 PostgreSQL / Docker 能力，因此我把后端切到了 **Python + FastAPI**，数据库在开发阶段改为了 **SQLite**。这一调整并不是单纯“为了省事”，而是基于交付目标做的务实选择。题目要求 48 小时内完成一个可访问、可演示的产品，且重点考察的是“如何用 AI 工具快速完成交付”，而不是纯工程语言本身。FastAPI 的优势在于：和 AI 生成代码协作效率高、接口定义直观、和 Pydantic/SQLAlchemy 配合紧密、适合快速搭建带文档的后端服务。聊天记录里也记录了这次转向：在发现 Go 不可用后，最终落地选择了 Python + FastAPI。

前端我选择了 **React 18 + TypeScript + Vite + Ant Design**。这套选择的原因比较明确。首先，React + TypeScript 是最成熟的后台管理系统技术组合，组件生态和 AI 生成代码的兼容性都很好。其次，Ant Design 对表格、筛选器、详情页、表单、状态标签、管理后台布局这类场景非常友好，能快速做出“运营工具”的感觉，而不是从零搭建 UI。

Vite 则让本地调试和开发体验更轻，便于快速启动和频繁迭代。聊天记录中，前端技术栈和运行方式也已明确，并通过 Vite 启动本地预览。

数据库方面，我最终采用的是**SQLite 作为本地开发与演示数据库**，并保留了未来迁移到 PostgreSQL 的结构设计。之所以接受 SQLite，是因为它足以支撑这次作业要求的规模：题目要求数据量不少于 100 条，而我最终构造了 200 条反馈数据，已经能够支持列表、筛选、看板和 AI 分析的基本演示。 另一方面，我也意识到：如果后续要做更完整的部署和扩展，尤其是做向量检索、异步任务、复杂统计、权限审计等，PostgreSQL 会是更合理的长期选择。

至于 AI 能力部分，我没有在这一版强行接入复杂的异步队列和嵌入服务，而是先把“AI 分类、AI 摘要、AI 建议”作为平台中的核心智能能力去设计，并在数据结构中保留了 `ai_category`、`ai_confidence` 等字段，便于后续接入真实模型输出。这个选择背后的原因是：题目要求的是一个“可用平台”，而不是一个完整的 MLOps 体系。如果一开始就把重点放在 Kafka、向量库、复杂聚类服务上，反而会牺牲交付速度和整体完成度。

综合来看，这次技术选型的原则是：**长期架构思路可以先进，但短期实现必须服从交付目标**。最终落地方案并不是我能想到的“最强架构”，但我认为它是当前约束下最适合完成题目的方案。

---

3. AI 协作过程

这次项目里，AI 不是单纯的代码生成器，而更像一个“高执行力的协作对象”。我和 Claude Code 的协作顺序大致分成五步：先做 PRD review，再做产品结构收敛，再做技术方案和执行计划，再开始搭建代码骨架，最后是不断基于实际结果修正数据、字段和页面逻辑。题目本身也明确说明，考察重点并不是代码必须自己手写，而是如何引导 AI、如何判断输出质量、以及如何解决问题。

第一阶段，我给 Claude 的关键指令更偏产品和架构，例如：帮我找出 PRD 中不合理、重复或不必要的部分；如果你来搭这个平台，你会怎么拆模块、怎么选技术栈、分几期实现。这一阶段 AI 的价值主要体现在“帮我发散”和“帮我补盲区”。例如它提醒我，原始 PRD 里模块间有重复、A/B Test 的边界过大、社媒抓取风险高、SLA 和通知机制定义不完整等。这些反馈很有价值，因为它逼着我重新确认：这个平台 V1 到底要解决什么问题，哪些能力虽然听起来高级，但会让产品失焦。

第二阶段，我让 AI 帮我生成 refined PRD 和 execution plan。这个阶段效果也比较好，因为 AI 很适合做结构化整理：把零散需求收束成产品文档，把模糊想法拆成模块、流程、字段和接口。这一步里我并没有直接接受 AI 的全部建议，而是反复人工改边界。例如我要求保留社媒反馈，只是降低复杂度；我要求把工单体系加回来；我也把 P0 响应时

间从 15 分钟调到 1 小时，把反馈与工单关系从 many-to-many 改回 1:1。聊天记录里能看到这些来回修改，说明 AI 给的是“候选答案”，最终拍板仍然是人。

第三阶段进入实现时，我给 Claude 的指令开始变得更具体，例如：先搭 FastAPI 后端骨架、定义数据模型和 Pydantic schema、生成 dashboard 服务和 API、补 auth 和 RBAC、再搭前端 scaffold。这个阶段 AI 在“重复性、高结构、模板化”的工作上效率很高，尤其是建目录、起项目、写基础 CRUD、补接口和页面框架这类任务。

第四阶段是我认为最关键的一步：**让 AI 处理真实反馈并生成模拟数据**。我没有直接让 AI“瞎编”200 条评论，而是先给它 20 条真实社媒反馈截图，让它提取文本、归类、补充上下文字段，再基于这些真实种子去生成 180 条扩展数据。这一部分的效果整体不错，因为真实样本帮助 AI 把语言风格、抱怨点、城市分布、高发场景都学得更贴近业务。聊天记录里，AI 先把 20 条真实反馈抽取成社媒信号记录，再进一步生成 200 条结构化反馈数据，并补全了题目要求之外的扩展字段。

但这个阶段也最能体现：**AI 的输出不能直接信**。我后续对数据做了很多人工校正。例如我发现 “乘驾体验”和“行驶体验”这两个标签语义重叠，于是要求重命名和重分类；我明确提出“定站点太少”“预估到达时间不准”应属于接驾体验；“USB 充电口不好找”“空调不会调”应归到新用户引导；我还删掉了 `experiment_group_id` 和 `driver_mode`，因为这些字段虽然技术上可以生成，但对当前产品没有帮助。聊天记录里，这些字段和分类的调整都经历了多轮修改。

所以，如果要总结哪类指令效果最好，我认为是两类。第一类是**边界清晰、输出格式明确**的指令，比如“生成 refined PRD”“补全 20 个字段”“按当前字段结构生成 200 条数据”。第二类是**局部修正型**指令，比如“删掉 driver_mode”“这类反馈应该归为接驾体验”“experiment_group_id 不需要”。相反，那些过于开放、边界不明确的任务，AI 往往会做得“很满”，甚至做过头。因此这次协作给我的一个很深体会是：和 AI 协作时，最重要的不是让它替你思考，而是你先把判断标准说清楚。

我判断 AI 输出质量的方法主要有三种。第一，**看是否符合业务逻辑**，例如分类是否合理、字段是否真的有用、SLA 是否现实。第二，**看是否服务于题目目标**，例如某个功能是不是偏题了、是不是让系统变重了。第三，**看输出之间是否一致**，比如 PRD、模型、schema、seed data、页面是否用的是同一套字段和分类体系。这一点很重要，因为 AI 特别容易出现“单点正确、全局不一致”的问题。

---

4. 问题与解决

整个项目过程中，我遇到的问题大致分为四类：需求边界问题、环境与技术实现问题、数据质量问题，以及前后端联调问题。

第一类是需求边界问题。最初的 PRD 更像一个“把想到的东西都放进来”的版本，里面混合了反馈平台、工单系统、舆情系统、实验分析平台等多个方向。AI 在 review 时帮助我快速指出了一些明显的逻辑问题和范围过大的部分，但真正的收敛还是靠我自己来做判断。例如，社媒抓取要不要保留、A/B Test 要不要做、工单要不要加回来、优先级到底是按情绪还是按安全来排，这些都不是 AI 能独立拍板的问题，而是产品判断题。最终我的做法是：保留题目主干，增加工单闭环，把社媒监控做轻，把实验能力降级为可选维度，而不是把它做成单独平台。

第二类是技术环境问题。最开始讨论的后端方案偏 Go / PostgreSQL，但实际环境里缺少 Go，且没有现成的 Postgres/Docker 条件，因此实现阶段只能转向 Python + FastAPI + SQLite。这个变化本身不算技术难题，但它会连带影响很多东西：ORM 写法、迁移方案、聚合 SQL、前后端联调方式等。聊天记录中，这一步是明确发生过的：在发现 Go 不可用后，实际开发路线转向了 Python，后续为了能本地跑通，又切到了 SQLite。 我认为这类问题 AI 很擅长给“可行替代方案”，但真正做取舍还是需要人判断：什么是必须保留的，什么是可以替代的。

第三类是数据问题，也是我投入精力最多的一块。最开始，AI 能把 20 条真实反馈抽取出来，并生成 180 条扩展数据，这说明它在“模仿风格”和“补全结构”上已经很强。但具体到业务细节时，问题很多。例如高评分文案和负面内容不一致，某些分类定义重叠，某些字段虽然能生成但没意义。我后续做了多轮人工校正：先把“行驶体验/乘驾体验”重新命名，再进一步把“定站点少、等车时间长、预计到达不准”统一归到接驾体验，把“USB/空调/座椅按摩不会用”归到新用户引导；之后又删除了 `driver_mode`，因为这个字段对当前产品不产生价值。聊天记录中，这些数据修正都经历了多次迭代，而且每次都是我先指出问题，再要求 AI 按新的业务规则修正。 这部分我会认为：AI 负责高效生成，我负责业务校验和最后定稿。

第四类是联调和运行问题。比如后端接口切换到 SQLite 后，一些统计逻辑要改成 SQLite 兼容写法；仪表盘接口曾经报 500；服务启动后有端口和重启问题；某些 dashboard 指标口径还发生过不一致，需要重新定义“负反馈”的判定逻辑。这些问题最终都通过重启服务、修复 SQL、调整统计逻辑解决了。 在这类问题上，AI 的强项是快速定位出错文件、尝试修改和回归验证，但是否接受这个修复、是否口径合理，还是需要我来确认。

如果要区分“哪些是 AI 解决的，哪些是我自己解决的”，我觉得可以这样理解：

AI 更适合解决的是“结构清楚、目标明确、可以快速试错”的问题，例如生成代码骨架、补接口、改字段、重写模板、修某个接口报错。

而我自己更重要的角色，是解决“判断题”：业务场景怎么理解、功能边界怎么划、字段到底有没有用、这类反馈到底归哪类、哪些功能该留、哪些该砍。

这个项目最能体现的，不是 AI 有多强，而是**人机协作如何分工**。

---

5. 反思与改进

如果重新做一次，我会做三件事上的调整。

第一，我会在更早的时候，把“平台边界”和“分类体系”一次性定得更清楚。回头看，后续很多返工其实不是代码问题，而是前面分类体系没有彻底定义清楚，比如接驾体验、新用户引导、驾驶行为之间的边界，或者某些字段到底是不是平台真正需要的。AI 能很快把东西做出来，但如果前面的概念没有定稳，后面就会不断返工。所以下次我会先花更多时间把“分类字典 + 字段字典 + 优先级规则”敲定，再开始生成代码和数据。

第二，我会把 AI 协作拆得更细，并尽量让每一步都能验证。现在回看，效果最好的阶段，都是我把任务切成了很明确的小块，例如“先改 PRD”“再生成 schema”“再修 seed data 分类”“再删掉无用字段”。而效果一般的时候，往往是我给了一个过大的任务，AI 虽然也能产出，但会把很多不必要的能力一起做进去。所以下次我会更系统地采用一种流程：先定义输入输出，再让 AI 生成，再人工审核，再决定是否进入下一步。这样比“先做出来再删”更稳。

第三，我会在时间允许的情况下，把两个进阶能力做得更完整。一个是**用户可选数据的自由问答**，让运营不仅能看摘要，还能直接围绕选中的反馈数据提问。另一个是**自动流转**，也就是不同类型的反馈自动分配给不同角色处理，例如驾驶行为问题给模型分析或算法产品，接驾体验给点位/运营，新用户引导类给运营策略，费用相关给商业化产品。这样平台会从“分析工具”进一步升级为“处理平台”。

这次项目最大的体会是：AI 并不会自动把模糊问题变成好结果，它只会非常快地把“你说的东西”实现出来。所以真正决定结果质量的，还是人的判断力，尤其是以下三点：

第一，能不能把问题定义清楚；

第二，能不能识别 AI 哪些输出虽然看起来合理、但其实偏题；

第三，能不能持续通过小步验证去修正方向。